Học Sâu (Deep Learning)

Chương 10: Diễn dịch và Trực quan hóa Mạng Tích chập (Interpreting what ConvNets learn)

Đại học Đông Á

August 14, 2026

Nội dung chương

1. 1. Vấn đề "Hộp Đen" trong Deep Learning
2. 2. Trực quan hóa Biểu diễn Trung gian
3. 3. Trực quan hóa Bộ lọc bằng Gradient Ascent
4. 4. Bản đồ Kích hoạt Lớp (Grad-CAM)
5. Tổng kết

Deep Learning có phải là Hộp Đen (Black Box)?

- Định kiến phổ biến cho rằng Mạng nơ-ron là những chiếc "Hộp đen", không thể giải thích được lý do nó đưa ra quyết định (Black Box Problem).
- Khả năng diễn dịch (**Interpretability**) là yêu cầu bắt buộc trong các lĩnh vực rủi ro cao:
  - **Y tế:** "Tại sao AI lại chẩn đoán bệnh nhân này bị ung thư?"
  - **Xe tự lái:** "Tại sao AI lại không nhận ra chiếc xe tải phía trước?"
- Thực tế: Định kiến "Hộp đen" **KHÔNG** đúng với Mạng Tích chập (ConvNets).
- ConvNets học các biểu diễn hình ảnh không gian 2D, do đó chúng ta có thể **trực quan hóa** những gì nó đang "nhìn" thấy.

Các phương pháp Trực quan hóa ConvNet

Trong chương này, chúng ta sẽ khảo sát 3 phương pháp cốt lõi nhất để mở khóa hộp đen ConvNet:

1. **Trực quan hóa Bản đồ đặc trưng trung gian (Intermediate Activations):** Giúp ta hiểu mô hình đang "nhìn" thấy những chi tiết nào của ảnh.
2. **Trực quan hóa Bộ lọc (Filter Visualization):** Sử dụng Gradient Ascent để vẽ ra các mẫu họa tiết (Pattern) lý tưởng mà mỗi Bộ lọc khát khao tìm kiếm nhất.
3. **Bản đồ Kích hoạt Lớp (Grad-CAM):** Trả lời câu hỏi "Tại sao AI đưa ra quyết định phân loại này?" thông qua một Bản đồ nhiệt (Heatmap) chỉ thẳng vào vùng ảnh quan trọng.

Kỹ thuật 1: Trực quan hóa Bản đồ Đặc trưng (Feature Maps)

- **Mục tiêu:** Hiểu cách các lớp Tích chập liên tiếp (Successive Layers) biến đổi và bóc tách một hình ảnh đầu vào.
- **Khái niệm Kích hoạt (Activations):** Đầu ra của một lớp Tích chập (bao gồm cả chiều không gian H, W và số kênh C) được gọi là Activation.
- Mỗi kênh (Channel) trong bản đồ đặc trưng hoạt động như một bộ dò mẫu (Pattern Detector) tương đối độc lập. Do đó ta sẽ xuất hình ảnh của từng Kênh ra màn hình để phân tích.

Ví dụ Đầu vào

- Ta đưa một bức ảnh con mèo (hoàn toàn mới, không có trong tập huấn luyện) vào mạng CNN phân loại Chó-Mèo đã được train ở Chương 8.
- Hãy xem cách các lớp Tích chập phản ứng (Activate) với bức ảnh này.



Hình 10.1: Hình ảnh con mèo thử nghiệm

Tiền xử lý ảnh gốc

Chúng ta dùng `keras.utils` để load ảnh gốc và chuyển nó thành mảng numpy Tensor sẵn sàng đưa vào mô hình.

```
import keras
import numpy as np
import matplotlib.pyplot as plt

img_path = "cat.jpg"
def get_img_array(img_path, target_size):
    # Opens the image file and resizes it
    img = keras.utils.load_img(img_path, target_size=target_size)
    # Turns the image into a float32 NumPy array
    array = keras.utils.img_to_array(img)
    # We add a dimension to transform our array into a "batch"
    array = np.expand_dims(array, axis=0)
    return array

img_tensor = get_img_array(img_path, target_size=(180, 180))
```

Xây dựng Mô hình Đa Đầu ra (Multi-output Model)

Để trích xuất các bản đồ đặc trưng ở giữa mạng (Intermediate), ta phải định nghĩa một Model đặc biệt nhận 1 đầu vào nhưng xuất ra **nhiều đầu ra** tương ứng với mọi lớp Conv2D.

```
from keras import layers

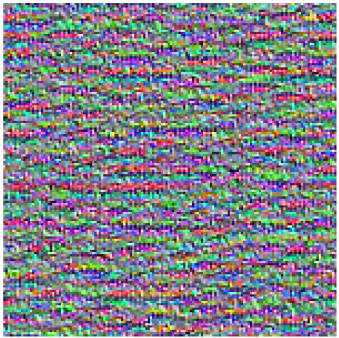
layer_outputs = []
layer_names = []
# Extracts the outputs of all Conv2D and MaxPooling2D layers
for layer in model.layers:
    if isinstance(layer, (layers.Conv2D, layers.MaxPooling2D)):
        layer_outputs.append(layer.output)
        layer_names.append(layer.name)

# Creates a model that will return these outputs
activation_model = keras.Model(inputs=model.input, outputs=layer_outputs)

# Returns a list of nine NumPy arrays
activations = activation_model.predict(img_tensor)
```

Kích hoạt của các Bộ lọc Đơn lẻ (Single Filter)

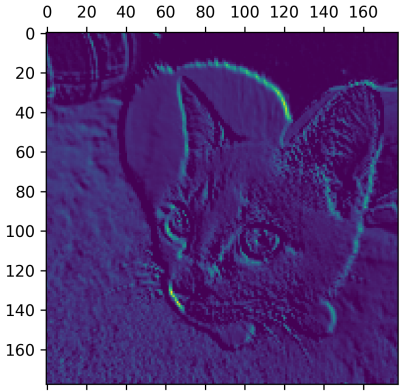
- Quan sát một kênh ở **Lớp đầu tiên** của mô hình. Kích thước Tensor: `(1, 178, 178, 32)`.
- Lớp này giữ lại gần như toàn bộ thông tin không gian chi tiết của con mèo.
- Kênh này đang hoạt động như một công cụ **Dò tìm Cạnh (Edge Detector)**, cực kỳ nhạy cảm với các đường viền nghiêng.



Hình 10.2: Đặc trưng Kênh số 4 (Lớp Conv đầu tiên)

Kích hoạt Kênh đặc biệt (Mắt mèo)

- Một kênh khác ở cùng lớp đầu tiên lại có nhiệm vụ hoàn toàn khác.
- Kênh số 5 này đặc biệt nhạy cảm với những **đốm sáng hình tròn** (như con ngươi của mắt mèo) và bỏ qua hầu hết các thông tin về lông hoặc cạnh tai.
- Quá trình chuyên môn hóa của các nơ-ron đã bắt đầu ngay từ lớp nông nhất.



Hình 10.3: Đặc trưng Kênh số 5 (Bắt dính Mắt mèo)

Trực quan hóa Toàn bộ Các kênh của Mạng (Phần 1)

Viết một vòng lặp `for` lồng nhau để lấy từng kênh ra và vẽ thành một lưới (Grid) ma trận lớn.

```
images_per_row = 16
for layer_name, layer_activation in zip(layer_names, activations):
    n_features = layer_activation.shape[-1]
    size = layer_activation.shape[1]
    n_cols = n_features // images_per_row
    display_grid = np.zeros(
        ((size + 1) * n_cols - 1, images_per_row * (size + 1) - 1)
    )
    for col in range(n_cols):
        for row in range(images_per_row):
            channel_index = col * images_per_row + row
            channel_image = layer_activation[0, :, :, channel_index].copy()
```

Trực quan hóa Toàn bộ Các kênh của Mạng (Phần 2)

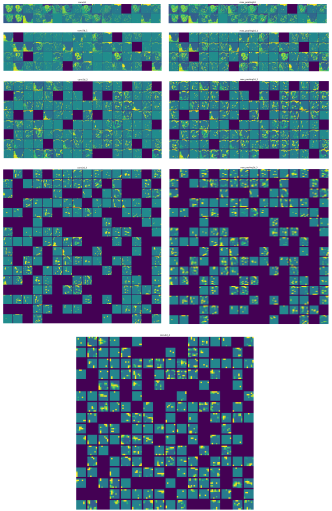
Ta cần Normalize giá trị của từng channel về phổ màu $0 - 255$ để màn hình máy tính có thể render được.

```
            # Normalizes channel values within the [0, 255] range.
            if channel_image.sum() != 0:
                channel_image -= channel_image.mean()
                channel_image /= channel_image.std()
                channel_image *= 64
                channel_image += 128
            channel_image = np.clip(channel_image, 0, 255).astype("uint8")

            # Places the channel matrix in the empty grid
            display_grid[
                col * (size + 1) : (col + 1) * size + col,
                row * (size + 1) : (row + 1) * size + row,
            ] = channel_image
```

Mạng Tích chập học được gì? (Lưới Kích hoạt)

- Khi tiến sâu hơn vào mạng, các bản đồ đặc trưng trở nên **thưa thớt (Sparse)** và **trừu tượng (Abstract)**.
- Chúng không còn chứa hình dáng vật lý của con mèo nữa, mà chuyển sang lưu giữ các Khái niệm (Ví dụ: "Có tai mèo trong vùng không gian này hay không?").
- Các ô trống đen xì nghĩa là bộ lọc đó hoàn toàn không tìm thấy mẫu mà nó cần.



Sự chưng cất thông tin (Information Distillation)

- Chúng ta vừa quan sát một hiện tượng phổ quát của Deep Learning: **Các tầng sâu sẽ học các đặc trưng trừu tượng hơn**.
- Mạng AI hoạt động như một **Ống chưng cất thông tin** (Information Distillation Pipeline).
- Các thông tin thừa thãi mang tính tiểu tiết (màu sắc lông, nền tường) sẽ bị lặp tức gạt bỏ. Những thông tin sống còn mang tính phân loại (có tai nhọn, có ria mép) sẽ được phóng đại và lưu giữ.
- Điều này giống hệt cách não bộ con người hoạt động: Khi nhìn 1 chiếc xe đạp, bạn chỉ nhớ "Đây là xe đạp" chứ không thể nhớ số nan hoa hay màu phanh xe.

Kỹ thuật 2: Gradient Ascent trên Không gian Ảnh

- **Mục tiêu:** Thay vì xem Bộ lọc phản ứng với ảnh thật như thế nào, ta hãy **tự tạo ra một bức ảnh ảo** (từ mảng nhiễu trắng ngẫu nhiên) sao cho Bộ lọc đó phản ứng cực đại.
- Bức ảnh ảo sinh ra chính là hiện thân cho "Mẫu lý tưởng" (Ideal Pattern) mà Bộ lọc đó đang khát khao tìm kiếm.
- Đây là thuật toán **Gradient Ascent (Leo đồi Gradient)**, đi ngược lại với Gradient Descent truyền thống.

Toán học của Gradient Ascent trên Pixel

Trong huấn luyện thông thường (Gradient Descent): Cố định Dữ liệu, **Cập nhật Trọng số (Weights)** để giảm thiểu Hàm Loss về cực tiểu.

$$W_{new} = W_{old} - \alpha \cdot \nabla_W L(W) \tag{1}$$

Trong Trực quan hóa Bộ lọc (Gradient Ascent): Cố định Trọng số, **Cập nhật Pixel của Ảnh (X)** để **tối đa hóa** Giá trị Kích hoạt của Bộ lọc (Activation A).

$$X_{new} = X_{old} + \alpha \cdot \nabla_X A(X) \tag{2}$$

Ta tính Đạo hàm của Đầu ra Bộ lọc theo Đầu vào Ảnh, sau đó cộng thêm đạo hàm đó vào Ảnh để làm nó rõ nét hơn qua từng epoch.

Định nghĩa Hàm Mất mát (Loss) cho Bộ lọc

Ta dùng cấu trúc Xception siêu mạnh để thực hành kỹ thuật này. Hãy định nghĩa một hàm `compute_loss` trả về trung bình giá trị kích hoạt của 1 channel.

```
import keras_hub
from keras import ops

model = keras_hub.models.Backbone.from_preset("xception_41_imagenet")
layer = model.get_layer(name="block3_sepconv1")
feature_extractor = keras.Model(inputs=model.input, outputs=layer.output)

def compute_loss(image, filter_index):
    activation = feature_extractor(image)
    # We avoid border artifacts by discarding the first 2 pixels
    filter_activation = activation[:, 2:-2, 2:-2, filter_index]
    # Returns the mean of the activation values for the filter
    return ops.mean(filter_activation)
```

Vòng lặp Gradient Ascent (TensorFlow)

Sử dụng `tf.GradientTape` để tính toán Gradient của Loss theo Image.

```python
import tensorflow as tf

@tf.function
def gradient_ascent_step(image, filter_index, learning_rate):
    with tf.GradientTape() as tape:
        # Explicitly watches the image tensor
        tape.watch(image)
        loss = compute_loss(image, filter_index)

    # Computes the gradients of the loss with respect to the image
    grads = tape.gradient(loss, image)
    # Applies the "gradient normalization trick"
    grads = ops.normalize(grads)
    # Moves the image a little bit in a direction that activates it
    image += learning_rate * grads
    return image
```

Khởi tạo Mảng Nhiễu Trắng ban đầu

Viết vòng lặp lặp lại 30 bước. Tại thời điểm t=0, bức ảnh là một mảng nhiễu trắng vô nghĩa.

```
def generate_filter_pattern(filter_index):
    iterations = 30
    learning_rate = 10.0
    image = keras.random.uniform(
        # Initialize an image tensor with random values centered on 0.5
        minval=0.4, maxval=0.6, shape=(1, 200, 200, 3)
    )

    # Repeatedly updates the values of the image tensor
    for i in range(iterations):
        image = gradient_ascent_step(image, filter_index, learning_rate)

    return image[0]
```

Xử lý hậu kỳ ảnh (Deprocess Image)

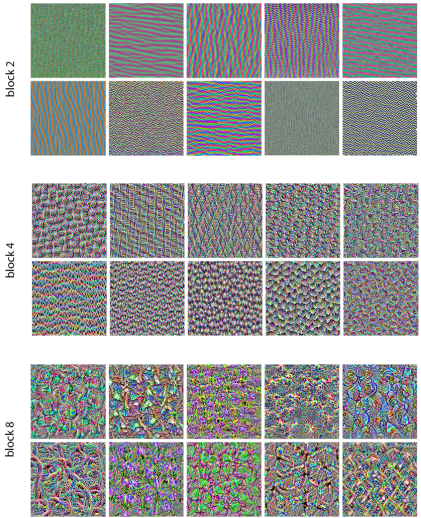
Sau vòng lặp 30 bước, ma trận thu được chứa các giá trị float rất dị thường. Ta phải ép nó về khoảng 0–255 chuẩn RGB.

```
def deprocess_image(image):
    # Normalizes image values
    image -= ops.mean(image)
    image /= ops.std(image)
    image *= 64
    image += 128
    image = ops.clip(image, 0, 255)

    # Center crop to avoid border artifacts
    image = image[25:-25, 25:-25, :]
    image = ops.cast(image, dtype="uint8")
    return ops.convert_to_numpy(image)
```

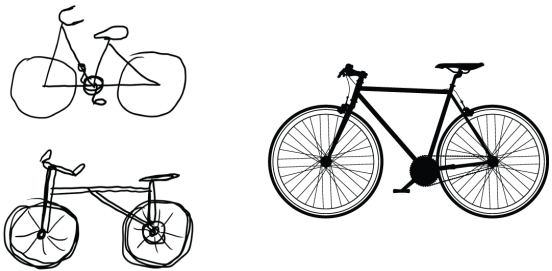
Quần thể các Mẫu Đặc trưng lý tưởng

- Kết quả của Gradient Ascent là những họa tiết kỳ ảo giống như ảo giác (Psychedelic).
- Các lớp nông (Shallow): Học màu sắc, vân gỗ, sóng nước.
- Các lớp giữa (Middle): Học kết cấu phức tạp hơn (chấm bi, sọc vằn, mạng nhện).



Lớp Sâu - Các Khái niệm Đời thực

- Đối với các khối ở cực sâu (như `block5_conv1` của VGG16/Xception), các họa tiết bắt đầu mang hình thù của vật thể thực.
- Chẳng hạn, bộ lọc này tổng hợp nên họa tiết hình elip đan chéo, rất giống nan hoa xe đạp (Bicycles), lông chim vũ hay vảy bò sát.
- Đây là bằng chứng tuyệt vời cho thấy Convolution Layers hoàn toàn có thể hiểu cấu trúc của vạn vật!



Hình 10.6: Họa tiết bộ lọc siêu sâu (Xe đạp / Vảy)

Kỹ thuật 3: Class Activation Map (CAM)

- **Mục tiêu:** Trả lời câu hỏi "Phần nào của bức ảnh đã khiến Mạng CNN đưa ra quyết định phân loại này?".
- Rất hữu ích cho **Phân tích lỗi (Error Analysis)** (Tại sao nhận diện sai?) và **Khám phá Y khoa** (Chỉ ra khối u nằm ở đâu trên ảnh X-Quang phổi).
- Phương pháp SOTA (State-of-the-art) là **Grad-CAM** (Gradient-weighted Class Activation Mapping).

Toán học của Grad-CAM

1. Tính Đạo hàm của xác suất Lớp dự đoán (Vd: African elephant y^c) theo toàn bộ Bản đồ đặc trưng (Feature Maps A^k) ở Lớp Tích chập **cuối cùng**.
2. Lấy Trung bình không gian (Global Average Pooling) của Đạo hàm này để ra Trọng số độ quan trọng α_k^c của Kênh k:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{i,j}^k} \tag{3}$$

3. Nhân chập Trọng số với Feature Maps tương ứng, rồi bọc qua hàm ReLU (chỉ lấy phần dương - Positive influence):

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right) \tag{4}$$

Đầu vào bài toán Phân tích Lỗi

- Ta đưa bức ảnh chụp hai con Voi châu Phi và con cái của chúng vào mô hình Xception chuẩn.
- Lớp `ImageClassifier` dự đoán: Voi châu Phi (African elephant) với xác suất cực cao.
- Câu hỏi đặt ra: Nó nhìn vào Voi mẹ hay Voi con để ra quyết định?



Hình 10.7: Ảnh Voi châu Phi đầu vào

Chuẩn bị Grad-CAM: Trích xuất Lớp Cuối

Ta tạo 2 Model phụ: Model 1 lấy đầu ra của Lớp tích chập cuối cùng. Model 2 chạy từ đầu ra đó đến thẳng lớp Softmax dự đoán.

```
# Model 1: From original image to the last conv layer feature map
last_conv_layer_name = "block14_sepconv2_act"
last_conv_layer = model.backbone.get_layer(last_conv_layer_name)
last_conv_layer_model = keras.Model(model.inputs, last_conv_layer.output)

# Model 2: From last conv layer feature map to predictions
classifier_input = last_conv_layer.output
x = classifier_input
for layer_name in ["pooler", "predictions"]:
    x = model.get_layer(layer_name)(x)
classifier_model = keras.Model(classifier_input, x)
```

Đạo hàm Xếp hạng Lớp (Class Gradients)

Dùng `tf.GradientTape` để lấy Gradient của Node dự đoán "African Elephant" đối chiếu với Bản đồ đặc trưng lớp dưới.

```
def get_top_class_gradients(img_array):
    last_conv_layer_output = last_conv_layer_model(img_array)
    with tf.GradientTape() as tape:
        tape.watch(last_conv_layer_output)
        preds = classifier_model(last_conv_layer_output)
        top_pred_index = ops.argmax(preds[0])
        top_class_channel = preds[:, top_pred_index]

    # Gradient of top class w.r.t the feature map
    grads = tape.gradient(top_class_channel, last_conv_layer_output)
    return grads, last_conv_layer_output

grads, last_conv_output = get_top_class_gradients(img_array)
```

Xây dựng Bản đồ Nhiệt (Heatmap)

Nhân chập Gradient (Trọng số tầm quan trọng của kênh) vào Feature Map để vẽ ra Heatmap thô.

```python
import numpy as np
import matplotlib.pyplot as plt

# Global average pooling the gradients
pooled_grads = np.mean(grads, axis=(0, 1, 2))
last_conv_output = last_conv_output[0].copy()

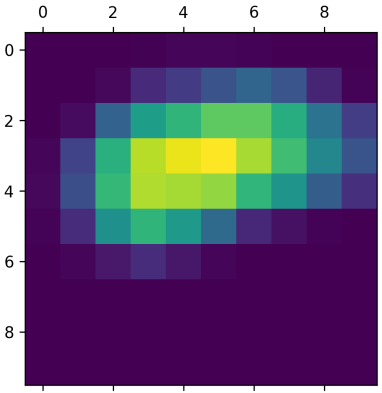
# Multiplies each channel by how important it is
for i in range(pooled_grads.shape[-1]):
    last_conv_output[:, :, i] *= pooled_grads[i]

# The channel-wise mean is our heatmap
heatmap = np.mean(last_conv_output, axis=-1)

# Apply ReLU manually
heatmap = np.maximum(heatmap, 0)
heatmap /= np.max(heatmap)
```

Bản đồ Nhiệt thô Grad-CAM

- Kết quả là một mảng Numpy 2D.
- Điểm có giá trị càng cao (Vùng sáng vàng chói) thì càng tác động lớn đến quyết định phân loại "Đây là con voi".



Hình 10.8: Bản đồ nhiệt thô Grad-CAM

Chồng bản đồ nhiệt (Superimposed Heatmap)

Ta áp dụng phổ màu JET (Xanh dương sang Đỏ) của `matplotlib` để nhuộm màu bản đồ, rồi áp lên trên ảnh gốc với hệ số trong suốt 0.4.

```
import matplotlib.cm as cm

img = keras.utils.load_img(img_path)
img = keras.utils.img_to_array(img)

heatmap = np.uint8(255 * heatmap)
jet = cm.get_cmap("jet")
jet_colors = jet(np.arange(256))[:, :3]
jet_heatmap = jet_colors[heatmap]

jet_heatmap = keras.utils.array_to_img(jet_heatmap)
jet_heatmap = jet_heatmap.resize((img.shape[1], img.shape[0]))
jet_heatmap = keras.utils.img_to_array(jet_heatmap)

# Superimposes the heatmap at 40% opacity
superimposed_img = jet_heatmap * 0.4 + img
```

Kết luận về thuật toán định vị trí

- **Kết luận đột phá:** Vùng đỏ rực tập trung chính xác vào **Đôi tai** của Voi con. Tai là đặc điểm sinh học định danh duy nhất phân biệt Voi châu Phi và Voi Ấn Độ.
- Mạng AI hoàn toàn không bị ngốc! Nó đã học đúng đặc điểm nhận dạng cốt lõi của sinh học phân tử thay vì học lỏm bầu trời hay cành cây.



Hình 10.9: Bản đồ Grad-CAM hiển thị tai Voi con

Tổng kết Chương 10

- Mạng Tích chập **KHÔNG** phải là Hộp đen. Ta hoàn toàn có thể "Nội soi" chúng dễ dàng bằng ngôn ngữ Python.
- **Intermediate Activations:** Phân tích tính trừu tượng tăng dần của Feature Maps (từ cạnh, góc đến mắt mèo).
- **Filter Visualization (Gradient Ascent):** Đạo hàm ngược lên Pixel ảnh để sinh ra ảnh ảo giác nhằm hiểu xem mỗi Bộ lọc khát khao tìm kiếm họa tiết nào.
- **Grad-CAM (Bản đồ Kích hoạt Lớp):** Trích xuất Heatmap vùng màu Đỏ để xác định vùng quyết định đóng góp lớn nhất vào Output dự đoán, là công cụ không thể thiếu trong X-Quang y tế học sâu.